

Review Tema 2

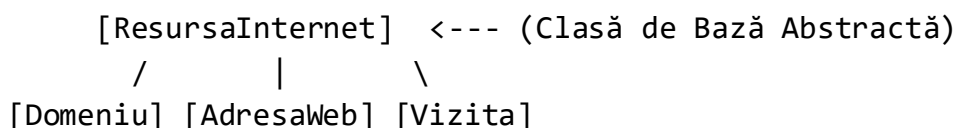
1. Introducere și Evaluare Generală

Proiectul „**InternetDocumentare**”, dezvoltat în C++, reprezintă un exemplu remarcabil de bune practici în programarea orientată pe obiecte (POO) și demonstrează o înțelegere profundă a conceptelor avansate din standardele moderne ale limbajului . Prin implementarea unei ierarhii de clase bine structurate—compusă din clasa de bază `ResursaInternet` și derivatele sale `Domeniu`, `AdresaWeb` și `Vizita`—codul reușește să modeleze fidel și elegant un domeniu de activitate real: monitorizarea, validarea, deduplicarea și prioritizarea resurselor web pe baza metricilor de engagement.

Fiecare componentă a proiectului este proiectată având în vedere modularitatea, siguranța codului în execuție și eficiența resurselor. De la utilizarea inteligentă a tehnicilor de tip *Resource Acquisition Is Initialization (RAII)* prin intermediul pointerilor inteligenți (`std::unique_ptr`), până la implementarea riguroasă a operatorilor de atribuire prin tehnica *Copy-and-Swap*, proiectul atinge un standard înalt de excelență academică și tehnică.

2. Analiza Arhitecturii și Ierarhiei de Mostenire

Elementul central al arhitecturii este clasa abstractă de bază `ResursaInternet`. Aceasta stabilește un contract clar pentru toate tipurile de resurse prin intermediul funcțiilor virtuale pure: `CalculeazaRelevanta()`, `Normalizeaza()`, `clone()` și `AfisareVirtuala()`.



Această ierarhie este concepută pentru a reflecta o relație de tip „*is-a*” (este o), dar proiectul merge un pas mai departe și integrează și o relație de compunere („*has-a*”). De exemplu, clasa `AdresaWeb` moștenește din `ResursaInternet`, dar în același timp deține o instanță a clasei `Domeniu`. Această combinație între moștenire și compunere oferă o flexibilitate extraordinară și împiedică duplicarea codului.

Designul funcțiilor din clasa de bază urmează parțial pattern-ul **NVI (Non-Virtual Interface)**. Prin funcțiile publice non-virtuale precum `Relevanta()`, `NormalizeazaResursa()` și `Afiseaza()`, clasa de bază controlează fluxul de execuție și apelează intern implementările virtuale private sau protejate ale claselor derivate. Acest lucru garantează că pașii generici de execuție pot fi monitorizați sau extinși fără a altera comportamentul specific al obiectelor derivate.

3. Utilizarea Excelentă a Standardului Modern C++ (STL și Smart Pointers)

Unul dintre punctele forte ale proiectului este utilizarea corectă și eficientă a containerelor și utilităților din biblioteca standard (STL). În loc să se bazeze pe gestionarea manuală a memoriei și pe pointeri de tip C (*raw pointers*), codul folosește `std::string` pentru manipularea textului și `std::vector` ca structură de date dinamică principală.

În clasa `ColectieResurse`, managementul memoriei este adus la cel mai înalt nivel prin utilizarea `std::vector<std::unique_ptr<ResursaInternet>>`.

- **Evitarea scurgerilor de memorie (Memory Leaks):** Prin utilizarea pointerilor unici (`std::unique_ptr`), memoria alocată dinamic pe heap pentru obiectele polimorfe este eliberată automat în momentul în care colecția iese din scope sau este ștearsă. Astfel, programul elimină complet riscul de scurgeri de memorie.
- **Transfer de proprietate sigur:** Utilizarea de `unique_ptr` împiedică copierea accidentală a resurselor și impune o semantică clară de proprietate unică asupra obiectelor conținute.

Mai mult, codul demonstrează o utilizare elegantă a algoritmilor STL. În funcția `SorteazaDupaRelevanta()`, se apelează algoritmul generic `std::sort` împreună cu o funcție lambda modernă:

```
C++
std::sort(resurse.begin(), resurse.end(),
    [](const std::unique_ptr<ResursaInternet> &A, const
std::unique_ptr<ResursaInternet> &B) {
    return A->Relevanta() > B->Relevanta();
});
```

Aceasta este o abordare expresivă, concisă și optimă, specifică programării C++ moderne.

4. Gestiunea Sigură a Memoriei: Copy-and-Swap și Clone

Un alt aspect tehnic demn de laudă este rezolvarea elegantă a problemelor de copiere a obiectelor polimorfe. În C++, copierea unui obiect printr-un pointer la clasa de bază poate duce la fenomenul de *slicing* (decupare). Proiectul previne acest lucru prin implementarea unei metode virtuale `clone()` în fiecare clasă derivată, permițând copierea polimorfică corectă.

În clasa `ResursaSelectata`, care deține un pointer la clasa de bază (`ResursaInternet*`), autorul implementează **idiomul Copy-and-Swap**. Acesta garantează „**Strong Exception Guarantee**” (obiectul rămâne neschimbat dacă o excepție este aruncată în timpul copierii) și elimină duplicarea codului între constructorul de copiere și operatorul de atribuire.

Constructorul de copiere folosește metoda `clone()` pentru a duplica resursa:

C++

```
ResursaSelectata::ResursaSelectata(const ResursaSelectata &R)
    : resursa(R.resursa != NULL ? R.resursa->clone() : NULL),
      nota(R.nota),
      prioritate(R.prioritate) {}
```

Operatorul de atribuire profită apoi de transmiterea prin valoare (care creează automat o copie) și de o metodă non-throwing `swap`:

C++

```
ResursaSelectata &ResursaSelectata::operator=(ResursaSelectata R) {
    swap(R);
    return *this;
}
```

Această implementare este considerată standardul de aur în C++ pentru scrierea unui cod robust, sigur la excepții și ușor de întreținut.

5. Polimorfism Dinamic, Validări și Tratarea Excepțiilor

Tratarea erorilor în cadrul proiectului nu se rezumă la simple mesaje afișate în consolă sau coduri de eroare returnate rudimentar. Arhitectura include o ierarhie dedicată de excepții, pornind de la clasa de bază `InternetException` (care moștenește din `std::exception`) și continuând cu excepții specifice: `DomeniuInvalidException`, `AdresaInvalidaException` și `VizitaInvalidaException`.

Acest design permite o granularitate excelentă la prinderea erorilor (`try-catch`), separând logica de business de mecanismul de raportare a problemelor. De exemplu, încercarea de a crea o adresă web cu un protocol nevalid (altul decât `http` sau `https`) aruncă instantaneu o excepție, împiedicând crearea unui obiect aflat într-o stare invalidă.

Un alt punct de excelență tehnică îl reprezintă utilizarea **`dynamic_cast`** pentru a realiza operațiuni de *downcasting* în siguranță. În metodele clasei `ResursaSelectata`, cum ar fi `ContineVizita()` și `NumarViziteDacaEsteVizita()`, programul verifică în mod dinamic dacă resursa stocată este de tipul derivat `Vizita`:

```
C++  
bool ResursaSelectata::ContineVizita() const {  
    return dynamic_cast<const Vizita *>(resursa) != NULL;  
}
```

Aceasta demonstrează o înțelegere clară a mecanismului RTTI (Run-Time Type Information) și oferă o metodă sigură de a interoga capacitățile obiectelor la runtime fără a risca un comportament nedefinit.

6. Curățarea Datelor și Algoritmi de Recomandare

Pe lângă structura sa tehnică impecabilă, proiectul oferă și soluții algoritmice interesante pentru procesarea datelor. Funcția `CuratareVizite` din clasa `Vizita` realizează deduplicarea resurselor accesate, un proces esențial în orice sistem real de monitorizare a traficului web. Atunci când sunt întâlnite domenii duplicate, sistemul nu doar că le elimină, ci agregă numărul de clicuri și concatenează datele calendaristice ale vizitelor.

În plus, funcția `RecomandareSite` aplică o formulă matematică personalizată bazată pe coeficienți ponderați pentru a calcula relevanța finală a unui site:

$$\text{Scor} = 0.50 \times \text{Clicuri} + 0.30 \times \text{ScorAdresă} + 0.20 \times \text{NumărDate}$$

Prin utilizarea acestei formule, proiectul transformă un simplu istoric de vizite într-un mic motor de analiză a datelor, demonstrând că este capabil să ofere valoare practică utilizatorului prin generarea unui top al celor mai relevante site-uri.

7. Concluzie

Proiectul „**InternetDocumentare**” este o dovadă excelentă de profesionalism în dezvoltarea software. Arhitectura sa respectă cu strictețe principiile fundamentale ale programării orientate pe obiecte:

- **Încapsulare:** Datele membre sunt private/protejate, accesibile doar prin getteri și setteri riguroși care validează inputul.
- **Moștenire și Polimorfism:** Ierarhia clară și utilizarea funcțiilor virtuale pure permit tratarea uniformă a resurselor internet.
- **Management Modern al Memoriei:** Utilizarea exclusivă a containerelor STL, a pointerilor inteligenți și a idiomului Copy-and-Swap garantează absența scurgerilor de memorie.

Dezvoltatorul acestui cod a reușit să îmbine teoria avansată din C++ cu aplicabilitatea practică, rezultând un cod curat, extensibil, ușor de citit și extrem de stabil. Este un proiect de nota 10, care poate servi oricând drept model de bune practici pentru orice programator C++.